Oracle C2M Batch Scheduling & Thread-Pool Management

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Batch Scheduling & Thread-Pool Management 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Oracle C2M Batch Scheduling & Thread-Pool Management 5

What C2M batch is . . . 5

Why this matters . . . 5

The C2M batch architecture . . . 5

Named objects this solution defines . . . 6

What this is not . . . 6

The deployment model . . . 7

2. Requirements Guide — Oracle C2M Batch Scheduling & Thread-Pool Management 7

2.1 A reachable C2M environment and product user . . . 7

2.2 Batch controls inventory (what runs) . . . 7

2.3 Thread-pool worker topology and capacity . . . 7

2.4 Named thread pools defined . . . 8

2.5 Scheduler decided and reachable (DBMS or enterprise) . . . 8

2.6 Database access for run status and monitoring . . . 8

2.7 JMX batch monitoring enabled (active-thread control) . . . 8

2.8 A non-production pilot environment and change control . . . 9

Checklist . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 9

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 10

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

3. Implementation Guide — Oracle C2M Batch Scheduling & Thread-Pool Management 11

Step 1 — Assess the current batch estate . . . 12

Step 2 — Define the named thread pools . . . 12

Step 3 — Configure the thread pool workers . . . 12

Step 4 — Map batch controls to the scheduler . . . 13

Step 5 — Build the schedule . . . 13

Step 6 — Encode dependencies . . . 13
Step 7 — Configure monitoring . . . 13
Step 8 — Pilot the cycle on non-production . . . 14
Step 9 — Validate against the pilot gate . . . 14
Step 10 — Expand to production . . . 14
5. Testing Guide — Oracle C2M Batch Scheduling & Thread-Pool Management 15
Test 1 — Workers advertise the standard pools . . . 15
Test 2 — A batch control submits into the right pool . . . 15
Test 3 — The nightly schedule fires . . . 15
Test 4 — Dependencies are honored . . . 15
Test 5 — Throughput and completion . . . 15
Test 6 — No stuck threads or backlog . . . 15
Test 7 — Live-thread control works . . . 15
Pass/fail gate . . . 16
6. Troubleshooting Guide — Oracle C2M Batch Scheduling & Thread-Pool Management 16
A. A batch control does not run . . . 16
B. Batch window overruns / throughput too low . . . 16
C. Dependencies not honored . . . 17
D. A worker node is missing a pool . . . 17
E. A thread is stuck . . . 17
F. Backlog does not drain . . . 17
G. JMX client cannot connect . . . 17
H. Diagnostics quick reference . . . 18
I. When to escalate . . . 18
7. Best Practices — Oracle C2M Batch Scheduling & Thread-Pool Management 18
8. Deliverables Summary — Oracle C2M Batch Scheduling & Thread-Pool Management 19
Documentation . . . 19
Named objects . . . 19
Automation scripts (WLST/Jython, shell & SQL — not PowerShell) . . . 20
What the buyer receives . . . 20
Outcome . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary — Oracle C2M Batch Scheduling & Thread-Pool Management

C2M does its heavy lifting in **batch** — billing, payment upload, meter-read processing, extracts, and hundreds of other background processes — and batch that is mis-scheduled or under-resourced quietly breaks the business: bills go out late, downstream jobs run before their upstream finished, and a stuck thread or growing backlog is discovered by a business user instead of by operations. This solution configures and manages C2M batch the repeatable way: **thread pool workers**, **batch scheduling and dependencies**, **scheduler integration** (Oracle DBMS scheduler or an enterprise scheduler), and **batch monitoring** — so jobs run on time and complete.

This is a complete, do-it-today package for an experienced C2M administrator. Every object is named once — the thread pools, the worker pool set, the nightly schedule, the dependency chain, the monitoring view — and reused across nodes, environments, and nightly cycles, so you configure it once and repeat it.

What you get

• A plain-language Solution Overview of the C2M batch architecture: batch controls, thread pool workers (`threadpoolworker`) and named pools, submission (`submitbatchREST.sh` / the batch daemon), scheduler integration (the `F1-DBMS` Program/Schedule/Job/Chain objects), and monitoring.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — a reachable environment, the batch-control inventory, worker capacity, named pools, a chosen scheduler, run- status access, and JMX monitoring.

• An Implementation Guide of precise, ordered steps: define named thread pools, configure the workers, map batch controls to the scheduler, build the nightly schedule, encode dependencies as chain rules, wire up monitoring, then pilot, validate throughput and completion, and expand.

• Five automation scripts (shell, WLST/Jython, and SQL — not PowerShell) for discovery, per-node validation, a run-status/backlog report, safe remediation (dry-run by default), and a scope/target helper — authentic to C2M and WebLogic administration.

• Testing and Troubleshooting guides with explicit pass/fail gates (pools up, jobs complete, no backlog), plus Best Practices for batch sizing and scheduling.

What this delivers

C2M batch that runs on time and completes — thread pools sized and named, the schedule and its dependencies correct, backlog and completion visible.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — proven on a non-production pilot before production, with a documented rollback for every production change.

Outcome

Reliable C2M batch: missed schedules, under-resourced pools, stuck jobs, and backlog replaced by sized thread pools, a correct schedule with real dependencies, and monitored completion — validated on a pilot before you scale.

1. Solution Overview — Oracle C2M Batch Scheduling & Thread-Pool Management

What C2M batch is

Oracle Utilities Customer to Meter (C2M) does most of its heavy lifting in **batch**: billing, payment upload, meter-read processing, To Do generation, workflow, extracts, and hundreds of other background processes. Each of these is a **batch control** — the framework object that names a background process, its parameters, and its defaults. A batch control is executed as one or more **threads**, and those threads run inside **thread pool workers** (`threadpoolworker`) — the JVMs that actually do the work. Getting C2M batch right means three things done together: sizing and running the right **thread pool workers**, scheduling the batch controls (and their **dependencies**) correctly, and **monitoring** completion and backlog. This solution configures and manages all three the repeatable way: named thread pools and schedules you define once and reuse everywhere.

> ***Version boundary.*** *Command names, script names, and object terminology below are from the Oracle Utilities Application Framework (OUAF) Server Administration Guide and C2M product documentation. Version-specific details (C2M 25.4 / minimum v2.9, WebLogic 14c, OUAF batch REST submission) are* ***cited or flagged "verify for your version"*** *— never invented. Some notes and patches are gated on My Oracle Support (MOS) and are pointed to, not reproduced. See* `global-requirements-oracle.md`*.*

Why this matters

Batch that is mis-scheduled or under-resourced quietly breaks the business. Under-resourced thread pools mean the nightly window overruns and bills go out late; over-resourced pools starve the online system or the database. Missing dependencies mean a downstream job (say, bill completion) runs before its upstream job (billing) finished, producing wrong or partial results. No monitoring means a stuck thread or a growing backlog is discovered by a business user, not by operations. A real solution fixes the cause, not the symptom: it sizes named thread pools deliberately, encodes dependencies in the scheduler, and makes backlog and completion visible — turning "did batch run?" from a morning surprise into a measured, monitored outcome.

The C2M batch architecture

Four moving parts, each with a named object you define and reuse:

1. **Batch control** — the process definition (for example the billing batch control, the payment

upload batch control). It carries the program name, default thread count, and parameters. You do not invent batch controls; C2M ships them. You *do* decide which pool each runs in and on what schedule.

1. **Thread pool worker (`threadpoolworker`)** — a JVM that offers one or more **named thread pools**

(for example `DEFAULT`, `ONLINE`, and a dedicated pool such as `BILLING`), each with a thread count. Batch threads execute inside these pools. Pools are how you partition capacity so heavy batch does not starve light or online-facing work.

1. **Submitter** — the mechanism that submits a batch control's threads to a pool. C2M supports the

REST submission path (`submitbatchREST.sh`, backed by the framework service that writes to the **Batch Job Submission** table and is polled by the batch daemon) and the legacy direct submitter (`submitjob.sh`). New work uses the REST/daemon path.

1. **Scheduler** — what decides *when* each batch control runs and in what order. C2M integrates with

the Oracle Database **DBMS scheduler** through framework `F1-DBMS` business services: a **DBMS Program** per batch control, a **DBMS Schedule** for frequency, a **DBMS Job** to run a program or chain, and a **DBMS Chain** to express **dependencies** (step B runs only if step A `SUCCEEDED`). Enterprise/third-party schedulers integrate through the same REST submission and monitoring surface.

Monitoring closes the loop: the framework **Batch Run Tree** / batch run status (in the database) and the JMX batch client (`jmxbatchclient.sh`) expose active threads, completion, and errors so a stuck or slow run is caught in operations, not in the business.

Named objects this solution defines

So the configuration is repeatable, this solution names its artifacts once and reuses them:

- **Thread pools** — `C2M Pool - Default - Batch`, `C2M Pool - Online - Batch`, and one dedicated

heavy pool `C2M Pool - Billing - Batch`, each with a documented thread count. (Pool names are your own; reuse them across every worker node.)

- **Thread pool worker config** — `C2M Worker - Node - Threadpools`, the standard set of pools each

worker node offers.

- **Schedule** — `C2M Schedule - Nightly - Batch Cycle`, the ordered nightly cycle of batch controls.

- **Dependency chain** — `C2M Chain - Nightly - Dependencies`, the DBMS chain that encodes which batch

control must complete before the next starts.

- **Monitoring view** — `C2M Monitor - Batch - Run Status`, the standing view of batch run status and

backlog.

What this is not

This solution configures and operates C2M's own batch framework. It does not rewrite C2M business logic, invent new batch controls, or replace the database. Where the included scripts touch live systems they **read** — the discovery, validation, and report scripts are read-only; remediation runs **dry-run by default**.

Any change to production thread-pool sizing, schedules, or dependencies goes through change control and is proven on a non-production environment first.

The deployment model

> *Assess the current batch estate and its bottlenecks, configure named thread pools plus the schedule and its dependencies, pilot the cycle on a non-production environment, validate throughput and completion against a gate, then expand to production. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*

2. Requirements Guide — Oracle C2M Batch Scheduling & Thread-Pool Management

Verify every requirement before configuring thread pools, schedules, or dependencies. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (licensing, certified stack, environments, WLST/shell/SQL tooling, change control) are in `global-requirements-oracle.md`.

2.1 A reachable C2M environment and product user

Description: A licensed C2M environment with its OUAF product home reachable, and the OS product user that owns it (the account under which `threadpoolworker`/`submitbatch` run), with the environment sourced via `splenviron.sh -e your-env`. Why: Every batch script (`threadpoolworker.sh`, `submitbatchREST.sh`, `jmxbatchclient.sh`) requires the environment variables `splenviron.sh` sets; without them the utilities cannot locate the domain, class path, or configuration. Verify: As the product user, run `splenviron.sh -e your-env` then run `OC2MBATCH-Discovery.sh` — it confirms the environment is sourced and the batch utilities are on the path. Implement-if-missing: Obtain the product-user credentials and the environment name; confirm the OUAF install is complete (`initialSetup`/`configureEnv` run) before proceeding.

2.2 Batch controls inventory (what runs)

Description: A known list of the C2M **batch controls** in scope — the background processes you must schedule (for example billing, payment upload, meter-read processing, To Do generation, extracts) — with each one's default thread count. Why: You cannot size thread pools or build a schedule without knowing which batch controls run, how heavy each is, and its default parallelism; the schedule and dependencies are defined per batch control. Verify: You can list the in-scope batch controls and their defaults; run `OC2MBATCH-Discovery.sh`, which reads batch-control metadata and recent run history from the C2M schema (read-only). Implement-if-missing: Compile the batch-control list from the C2M batch schedule/run history and the business owners; record each control's purpose and default threads.

2.3 Thread-pool worker topology and capacity

Description: The planned **thread pool worker** nodes (which hosts run `threadpoolworker`), and the CPU/memory/JVM-heap headroom on each, so pools can be sized without starving the online tier or the database. Why: Thread count is only useful if the host and JVM can carry it; over-committing threads causes GC thrashing, database contention, and overrun windows — under-committing wastes the batch

window. Verify: For each planned worker host you can state cores, memory, and the JVM heap the worker will use; run `OC2MBATCH-Discovery.sh` to report the currently configured pools and thread counts. Implement-if-missing: Confirm the worker hosts and their resource headroom with the infrastructure team; document the heap sizing per node before assigning thread counts.

2.4 Named thread pools defined

Description: The named thread pools this solution standardizes — `C2M Pool - Default - Batch`, `C2M Pool - Online - Batch`, and the dedicated heavy pool `C2M Pool - Billing - Batch` — each with a documented thread count, offered consistently by every worker node (`C2M Worker - Node - Threadpools`). Why: Named, consistent pools are what let you partition capacity (keep heavy billing off the online pool) and reuse the same worker configuration across nodes and environments. Verify: Each worker node offers the same pool names; run `OC2MBATCH-Validation.sh`, which checks that every node advertises the standard pool set with the expected thread counts. Implement-if-missing: Define the pool names and thread counts (see the Implementation Guide) and apply them to each worker node's startup (`threadpoolworker -p ...`) or `threadpoolworker.properties`.

2.5 Scheduler decided and reachable (DBMS or enterprise)

Description: A chosen scheduler — the Oracle Database **DBMS scheduler** (via the framework `F1-DBMS` business services: DBMS Program, Schedule, Job, Chain) or an enterprise/third-party scheduler submitting through `submitbatchREST.sh` — with the access to define programs, jobs, and chains. Why: A scheduler is what turns a list of batch controls into an ordered, time-based cycle with dependencies; without one, batch is submitted by hand and dependencies are enforced by hope. Verify: You can define (or already have) a DBMS Program for a representative batch control, or invoke `submitbatchREST.sh -b your-batch-control` successfully; run `OC2MBATCH-Validation.sh` (checks scheduler reachability signals). Implement-if-missing: Choose the scheduler; for DBMS, confirm the schema has the `F1-DBMS` services and the privileges to create scheduler objects; for enterprise, confirm REST submission works.

2.6 Database access for run status and monitoring

Description: Read access to the C2M schema tables that hold **batch run status** (batch run, batch run thread, batch instance) so run history, completion, and backlog can be reported. Why: Monitoring closes the loop; without run-status access there is no `C2M Monitor - Batch - Run Status` view and a stuck or backlogged run is invisible to operations. Verify: You can query batch run status for a recent run; run `OC2MBATCH-Report.sql` via SQL*Plus/SQLcl against the schema (read-only) and confirm it returns recent runs. Implement-if-missing: Grant the reporting account SELECT on the batch run-status tables (or use the product user); confirm SQL connectivity per the global requirements.

2.7 JMX batch monitoring enabled (active-thread control)

Description: The batch JMX interface enabled on the worker nodes (the RMI/JMX port populated, e.g. `BATCH_RMI_PORT`) so `jmxbatchclient.sh` can list, filter, and — when authorized — cancel active threads. Why: Run-status tables show completed and in-progress runs; JMX is how operations sees and controls *live* threads (a stuck thread, a runaway pool) without bouncing the worker. Verify: `jmxbatchclient.sh -j your-jmx-url -s` returns an active-thread summary; `OC2MBATCH-Validation.sh` reports whether the JMX port is configured. Implement-if-missing: Set the

JMX/RMI port in the worker configuration and restart the worker in a change window; confirm the client connects.

2.8 A non-production pilot environment and change control

Description: A non-production C2M environment to size pools, build the schedule, and prove the dependency chain before touching production, plus change control and backups (domain config, `threadpoolworker.properties`, scheduler objects) per the global requirements. Why: Thread-pool sizing and dependency changes are high-impact; the model is Assess → Configure → Pilot → Validate → Expand with a documented rollback. Verify: A non-production environment is available and a backup/rollback path for the worker config and scheduler objects is confirmed. Implement-if-missing: Stand up or obtain the non-production environment and establish the backup and change window before any production change.

Checklist

- C2M environment reachable, product user, `splenviron.sh -e your-env` sources cleanly — `OC2MBATCH-Discovery.sh`
- Batch controls inventory with default threads — `OC2MBATCH-Discovery.sh`
- Worker topology and capacity known
- Named thread pools defined — `C2M Pool - Default/Online/Billing - Batch`, `OC2MBATCH-Validation.sh`
- Scheduler chosen and reachable (DBMS `F1-DBMS` or enterprise via `submitbatchREST.sh`)
- DB access to batch run-status tables — `OC2MBATCH-Report.sql`
- JMX batch monitoring enabled — `jmxbatchclient.sh`, `OC2MBATCH-Validation.sh`
- Non-production pilot + change control and backups

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL

connectivity to the schema.

- **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

- **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.
- **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.
- **How to verify:** Confirm a backup/restore path and a change window/approval for production.
- **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

- **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.
- **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.
- **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.
- **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M Batch Scheduling & Thread-Pool Management

Follow these steps in order. This guide configures C2M batch: named thread pool workers first, then the schedule and its dependencies, then scheduler integration, then validation of throughput and completion — pilot-first on a non-production environment, then expand to production under change control. Object names are fixed — create them with the exact names shown and reuse them everywhere.

> *Model: assess the estate, configure pools and the schedule, prove it on a non-production pilot, validate against the gate, then expand to production. Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*
>
> *All commands assume the environment is sourced first:* `splenviron.sh -e your-env` *(run as the product user). Command and script names (*`threadpoolworker`*,* `submitbatchREST.sh`*,* `jmxbatchclient.sh`*,* `F1-DBMS` *services) are from the OUAF Server Administration Guide — verify the exact flags for your version.*

Step 1 — Assess the current batch estate

1. Source the environment (`splenviron.sh -e your-env`) and run `OC2MBATCH-Discovery.sh`. It captures:

the in-scope batch controls and their default thread counts, the currently configured thread pools and thread counts on each worker node, and recent batch run history (durations, failures) from the C2M schema (read-only).

1. Identify the bottleneck controls (longest-running, most-failed) and the overrun points in the

nightly window. Record gaps against the Requirements Guide checklist. Do not proceed past a missing environment, missing scheduler decision, or missing run-status access — implement those first.

Step 2 — Define the named thread pools

1. Decide capacity per pool from the worker topology (Requirement 2.3). Standardize three pools:

• `C2M Pool - Default - Batch` — general batch controls (name it `DEFAULT` in the worker config).

• `C2M Pool - Online - Batch` — light, online-adjacent processes kept off the heavy pool (name it `ONLINE`).

• `C2M Pool - Billing - Batch` — the dedicated heavy pool for billing and other long controls (name it `BILLING`).

1. Assign a documented thread count to each pool per node, sized to the host's cores/heap so total

threads across pools do not over-commit the JVM or the database.

Step 3 — Configure the thread pool workers

1. On each worker node, configure the standard pool set `C2M Worker - Node - Threadpools` — every node

offers the same pool names so the configuration is portable. Set the pools either at startup or in `threadpoolworker.properties`:

```
# every worker node offers the same named pools with node-sized thread counts
threadpoolworker.sh -p DEFAULT=5,ONLINE=3,BILLING=8
```

(`-p name=count,...` offers the named pools; size the counts per node. Verify flag syntax for your version in the Server Administration Guide.)

1. Designate the scheduler-daemon node if you use the REST/daemon submission path (the node that polls

the **Batch Job Submission** table), per the version's `-d` daemon option.

1. Enable JMX on each worker (populate the RMI/JMX port, e.g. `BATCH_RMI_PORT`) so `jmxbatchclient.sh`

can monitor and control live threads.

1. Start the workers in a change window and confirm each advertises the standard pools.

Step 4 — Map batch controls to the scheduler

1. For each in-scope batch control, define a scheduler **program**:

• **DBMS scheduler:** create a **DBMS Program** per batch control via the framework `F1-DBMS` business services (one program per batch control that needs scheduling).

• **Enterprise/third-party scheduler:** define the job to invoke `submitbatchREST.sh -b your-batch-control -c threads` (REST submission writes to the Batch Job Submission table, which the batch daemon polls and executes).

1. Set each program to submit its batch control into the correct **pool** (heavy controls to `BILLING`, light to `ONLINE`, the rest to `DEFAULT`) with the intended thread count.

Step 5 — Build the schedule

1. Create the schedule `C2M Schedule - Nightly - Batch Cycle`:

• **DBMS scheduler:** define a **DBMS Schedule** (the nightly frequency) and a **DBMS Job** to run the cycle.

• **Enterprise scheduler:** define the equivalent time trigger for the cycle.

1. Order the batch controls into the nightly cycle from the assessment (Step 1) so heavy controls run in the widest part of the window.

Step 6 — Encode dependencies

1. Create the dependency chain `C2M Chain - Nightly - Dependencies`:

• **DBMS scheduler:** define a **DBMS Chain** whose steps reference the DBMS Programs, with rules that gate each step on its predecessor (for example a step starts only on `predecessor SUCCEEDED` — the framework's documented chain-rule form). This is how "bill completion runs only after billing succeeds" is enforced, not assumed.

• **Enterprise scheduler:** express the same predecessor→successor dependencies as the scheduler's job dependencies.

1. Verify every hard dependency from the business (billing before completion, upload before posting, etc.) is a rule in the chain.

Step 7 — Configure monitoring

1. Stand up the monitoring view `C2M Monitor - Batch - Run Status` from the batch run-status tables:

run `OC2MBATCH-Report.sql` (read-only) to render recent run status, durations, failures, and backlog.

1. Confirm live-thread visibility: `jmxbatchclient.sh -j your-jmx-url -s` for an active-thread summary

and `-d` for detail. This is the operational view during the run.

Step 8 — Pilot the cycle on non-production

1. On the non-production environment, run the whole nightly cycle end to end through the scheduler

using the pools, schedule, and dependency chain.

1. Run `OC2MBATCH-Validation.sh` to confirm the pools are advertised as expected, the scheduler is

reachable, and the run-status/JMX signals are present.

1. Run `OC2MBATCH-Report.sql` to capture throughput (records/sec, thread completion) and confirm every

control completed and the chain honored its dependencies.

Step 9 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when: every worker advertises the standard pools;

the cycle ran end to end within the window; every dependency in the chain was honored; and the run-status report shows all controls completed with no stuck threads and no backlog.

1. Remediate any configuration gaps with `OC2MBATCH-Remediation.sh` (dry-run by default — it prints

the intended change; add the apply flag only in a change window) before re-running validation.

Step 10 — Expand to production

1. After the pilot passes the gate, apply the same pools (`C2M Worker - Node - Threadpools`), schedule

(`C2M Schedule - Nightly - Batch Cycle`), and dependency chain (`C2M Chain - Nightly - Dependencies`) to production under change control, with backups of the worker config and scheduler objects taken first and a documented rollback ready.

1. Re-run `OC2MBATCH-Validation.sh` and `OC2MBATCH-Report.sql` in production after the first cycle;

keep the monitoring view running so backlog and completion stay visible night over night.

Order of operations

```
Assess -> Named pools -> Configure workers -> Map controls to scheduler -> Build schedule -> Encode dependencies -> Monitoring -> Pilot cycle -> Validate gate -> Expand to production
```

5. Testing Guide — Oracle C2M Batch Scheduling & Thread-Pool Management

Run after the pilot cycle on the non-production environment (Implementation Step 8) and again after production expansion. Each test confirms one part of the configured batch cycle works end to end. Source the environment first: `splenviron.sh -e your-env`.

Test 1 — Workers advertise the standard pools

Do: On each worker node, confirm the pools are offered; run `OC2MBATCH-Validation.sh`. Expected: Every node advertises `DEFAULT`, `ONLINE`, and `BILLING` (the `C2M Worker - Node - Threadpools` set) with the expected thread counts — no node missing a pool.

Test 2 — A batch control submits into the right pool

Do: Submit a representative light control and the heavy billing control; observe with `jmxbatchclient.sh -j your-jmx-url -d`. Expected: Threads for the heavy control run in `BILLING`, light control in `ONLINE`/`DEFAULT` — heavy batch is not sharing the online pool.

Test 3 — The nightly schedule fires

Do: Let `C2M Schedule - Nightly - Batch Cycle` trigger (or trigger it manually via the DBMS Job / enterprise scheduler). Expected: The cycle starts on schedule and the batch controls submit in the defined order.

Test 4 — Dependencies are honored

Do: Run the cycle through `C2M Chain - Nightly - Dependencies`; inspect run status with `OC2MBATCH-Report.sql`. Expected: Each dependent control starts only after its predecessor completed successfully (for example bill completion after billing `SUCCEEDED`); no dependent control ran early on a failed predecessor.

Test 5 — Throughput and completion

Do: Run the full cycle; capture `OC2MBATCH-Report.sql`. Expected: Every in-scope batch control completes within the window; per-thread completion and records/sec are recorded; no thread ended in error.

Test 6 — No stuck threads or backlog

Do: During and after the run, check `jmxbatchclient.sh -j your-jmx-url -s` and `OC2MBATCH-Report.sql`. Expected: No thread is stuck active past its expected duration; the Batch Job Submission backlog drains to zero after the cycle; `C2M Monitor - Batch - Run Status` shows all-complete.

Test 7 — Live-thread control works

Do: On a safe, cancellable test run, use `jmxbatchclient.sh -j your-jmx-url -f pattern -c` to cancel a matching thread (non-production only). Expected: The targeted thread cancels cleanly and the worker keeps running — operations can control a runaway thread without bouncing the node.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass on non-production: pools advertised, schedule fires, dependencies honored, cycle completes in-window, no stuck threads/backlog
Expand pass	Tests 1-6 pass in production after the first cycle, and Test 7 (live-thread control) is verified in non-production before go-live

6. Troubleshooting Guide — Oracle C2M Batch Scheduling & Thread-Pool Management

Use `OC2MBATCH-Discovery.sh` and `OC2MBATCH-Validation.sh` for configuration and coverage state, `OC2MBATCH-Report.sql` for the run-status/backlog view, `jmxbatchclient.sh` for live threads, and `OC2MBATCH-Remediation.sh` (dry-run by default) for safe fixes to pool/submission gaps. Source the environment first: `splenviron.sh -e your-env`. MOS-gated notes are pointed to, not reproduced.

A. A batch control does not run

Symptom	Cause	Remediation
Control never starts	Scheduler program/job missing	Confirm the DBMS Program/Job (or enterprise job) exists for the control; re-check with `OC2MBATCH-Validation.sh`
Control queued, never picked up	Batch daemon not running / wrong node	Confirm the daemon node is up and polling the Batch Job Submission table; `OC2MBATCH-Report.sql` shows the backlog growing
REST submission fails	`submitbatchREST.sh` args or REST service	Verify `-b your-batch-control` and thread flag; confirm the framework REST submit service is reachable for your version

B. Batch window overruns / throughput too low

Symptom	Cause	Remediation
Cycle runs past the window	Heavy control under-threaded or in the wrong pool	Move it to `C2M Pool - Billing - Batch`; raise its thread count within host headroom; re-measure with `OC2MBATCH-Report.sql`

Symptom	Cause	Remediation
Threads run but slowly	Database contention from over-committed threads	Reduce total threads across pools/nodes; check DB waits with the DBA — more threads can be slower
Online tier degrades during batch	Heavy batch sharing the online pool	Ensure heavy controls use `BILLING`, not `ONLINE`; re-validate pool assignment

C. Dependencies not honored

• A dependent control ran before its predecessor finished. The dependency was timing-based, not a chain rule. Encode it in `C2M Chain - Nightly - Dependencies` so the successor starts only on `predecessor SUCCEEDED`; re-run the cycle and confirm order with `OC2MBATCH-Report.sql`.

D. A worker node is missing a pool

Symptom	Cause	Remediation
Validation flags a missing pool	Node started without the standard `-p` set	Restart the worker with `DEFAULT=..,ONLINE=..,BILLING=..` (the `C2M Worker - Node - Threadpools` set) or set it in `threadpoolworker.properties`; `OC2MBATCH-Remediation.sh` shows the intended fix (dry-run)

E. A thread is stuck

• A thread stays active far past its expected duration. List it with `jmxbatchclient.sh -j your-jmx-url -d`, filter with `-f pattern`, and cancel the specific thread with `-c` (non-production first; production only under change control). If threads repeatedly stick on one control, capture the run and review the control with the application team — a stuck thread past all fixes is a code/data issue, not a config one.

F. Backlog does not drain

• The Batch Job Submission backlog stays non-zero after the cycle. Confirm the daemon node is polling, the target pool has free threads (`jmxbatchclient.sh -s`), and no control is failing-and-retrying. Run `OC2MBATCH-Report.sql` to see which control is backlogged, then re-size or re-sequence it.

G. JMX client cannot connect

• `jmxbatchclient.sh` fails to reach a worker. Confirm the RMI/JMX port (e.g. `BATCH_RMI_PORT`) is

populated on that worker and reachable, and that you passed the correct `-j your-jmx-url`; `OC2MBATCH-Validation.sh` reports whether the port is configured. Restart the worker in a change window if the port was not set.

H. Diagnostics quick reference

```
splenviron.sh -e your-env                          # source the environment first
./OC2MBATCH-Discovery.sh                            # controls, pools/counts, recent run history (read-only)
./OC2MBATCH-Validation.sh                           # pools advertised, scheduler + JMX signals (read-only)
sqlplus user@db @OC2MBATCH-Report.sql               # run status, throughput, backlog (read-only)
jmxbatchclient.sh -j your-jmx-url -s                # live active-thread summary
./OC2MBATCH-Remediation.sh                          # dry-run by default; add the apply flag in a change win
```

I. When to escalate

If the pools are advertised, the scheduler and daemon are healthy, dependencies are chain-encoded, and a control still overruns or fails after re-sizing and remediation, capture `OC2MBATCH-Validation.sh`, `OC2MBATCH-Report.sql`, and the `jmxbatchclient.sh -d` output and review with the DBA and application team. For product defects, consult the version's My Oracle Support notes and open a service request — gated content is pointed to, not reproduced here.

7. Best Practices — Oracle C2M Batch Scheduling & Thread-Pool Management

- Partition capacity with named pools. Keep heavy controls in `C2M Pool - Billing - Batch` and light

or online-adjacent work in `C2M Pool - Online - Batch`; a single shared pool lets one long billing run starve everything else. Offer the same pool names on every node (`C2M Worker - Node - Threadpools`) so config is portable.

- Size threads to the host and the database, not to a wish. Total threads across all pools on a node

must fit the JVM heap and the CPU, and the aggregate across nodes must not overwhelm the database — over-committing threads causes GC thrashing and DB contention that make batch *slower*, not faster. Start conservative, measure with `OC2MBATCH-Report.sql`, then raise.

- Match thread count to the batch control's own default and data volume. A control's shipped default

thread count is a starting point; tune per control against its record volume rather than applying one number everywhere.

- Prefer the REST/daemon submission path for new work. `submitbatchREST.sh` writes to the Batch Job

Submission table that the batch daemon polls — it is the supported modern path; keep the legacy direct submitter only where you must.

- Encode every hard dependency as a chain rule, never as timing. Use `C2M Chain - Nightly -

Dependencies so a successor starts on predecessor SUCCEEDED`, not "20 minutes later and hope" — gap-timing dependencies break the first slow night.

- Schedule the heavy controls into the widest part of the window. Put billing and other long controls

where they have room; sequence lighter controls around them so the window does not overrun.

- Monitor live and after. Watch active threads during the run with `jmxbatchclient.sh` and completion

and backlog after with `OC2MBATCH-Report.sql` — a growing Batch Job Submission backlog is the earliest warning that pools are under-sized or a control is stuck.

- Keep run-status visible every night. Make `C2M Monitor - Batch - Run Status` a standing operational

view so a missed or failed control is caught by operations at completion, not by a business user the next morning.

- Change pools, schedules, and dependencies under change control only. Back up

`threadpoolworker.properties` and the scheduler objects, prove the change on non-production first, and keep a documented rollback — batch config changes are high-impact.

- Prove throughput on a pilot before you scale. Run the whole cycle on non-production and pass the

gate (pools advertised, dependencies honored, completes in-window, no backlog) before touching production.

8. Deliverables Summary — Oracle C2M Batch Scheduling & Thread-Pool Management

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M batch architecture: thread pool workers, batch controls, scheduling, dependencies, monitoring (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: named pools, configure workers, scheduler mapping, schedule, dependencies, validate
5. Testing Guide	Validation procedures with pass/fail gates (pools up, jobs complete, no backlog)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for batch sizing and scheduling

Named objects

Object	Name
Default thread pool	`C2M Pool - Default - Batch` (`DEFAULT`)
Online thread pool	`C2M Pool - Online - Batch` (`ONLINE`)
Heavy/billing thread pool	`C2M Pool - Billing - Batch` (`BILLING`)
Worker pool set	`C2M Worker - Node - Threadpools`
Nightly schedule	`C2M Schedule - Nightly - Batch Cycle`
Dependency chain	`C2M Chain - Nightly - Dependencies`
Monitoring view	`C2M Monitor - Batch - Run Status`

Automation scripts (WLST/Jython, shell & SQL — not PowerShell)

Script	What it does
`OC2MBATCH-Discovery.sh`	Reports batch controls and defaults, configured thread pools/counts per worker node, and recent run history from the C2M schema (read-only)
`OC2MBATCH-Validation.sh`	Confirms each worker advertises the standard pools, scheduler reachability, and run-status/JMX signals (read-only)
`OC2MBATCH-Report.sql`	Renders batch run status, durations, failures, throughput, and Batch Job Submission backlog from the schema (read-only)
`OC2MBATCH-Remediation.sh`	Aligns pool config / submission gaps to the standard; dry-run by default, applies only with an explicit flag in a change window
`OC2MBATCH-ScopeHelper.sh`	Builds the in-scope batch-control / worker-node target list for pilot and expansion

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five automation scripts (shell / WLST-Jython / SQL) in a single ZIP.
- A repeatable configuration following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

C2M batch that runs on time and completes — thread pools sized and named, the schedule and its dependencies correct, backlog and completion visible — proven on a non-production pilot before it is expanded to production under change control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MBATCH-Discovery.sh

Purpose: Capture the current-state batch estate for an Oracle Utilities C2M environment: the in-scope batch controls and their default thread counts, the thread pools and thread counts currently configured on each worker node, and recent batch run history (durations, failures) from the C2M schema. Read-only — it changes nothing. Run it before sizing pools or building a schedule (Implementation Step 1). HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Discovery.sh Optional: set DB_CONN=user/pass@your-db to include run-history from the schema. EXPECTED RESULTS Prints: environment sourced OK and batch utilities on PATH; the configured thread pools/counts from threadpoolworker.properties; and (if DB_CONN is set) the recent batch run history and per-control defaults. PASS = environment sourced and pools listed. FAIL = environment not sourced or utilities not found — fix before continuing. NOTES White-label, repeatable discovery. Read-only where it reads live systems: it only SELECTs from the schema and reads config files. Command/script names are from the OUAF Server Administration Guide — verify exact names/flags for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Discovery.sh Optional: set DB_CONN=user/pass@your-db to include run-history from the schema. EXPECTED RESULTS Prints: environment sourced OK and batch utilities on PATH; the configured thread pools/counts from threadpoolworker.properties; and (if DB_CONN is set) the recent batch run history and per-control defaults. PASS = environment sourced and pools listed. FAIL = environment not sourced or utilities not found — fix before continuing. NOTES White-label, repeatable discovery. Read-only where it reads live systems: it only SELECTs from the schema and reads config files. Command/script names are from the OUAF Server Administration Guide — verify exact names/flags for your version.

OC2MBATCH-Remediation.sh

Purpose: Bring a worker node's thread-pool configuration into line with the standard set (DEFAULT, ONLINE, BILLING — the C2M Worker - Node - Threadpools set) when validation flags a missing or mis-sized pool. DRY-RUN by default: it prints the exact change it WOULD make (the threadpoolworker startup line / property edit) and changes nothing. It applies only when explicitly asked, and only intended for a change window. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Remediation.sh # DRY-RUN: prints intended change only ./OC2MBATCH-Remediation.sh --apply # applies (change window only) Override sizing: DEFAULT_THREADS=5 ONLINE_THREADS=3 BILLING_THREADS=8 ./OC2MBATCH-Remediation.sh EXPECTED RESULTS DRY-RUN prints the standard pool line and any drift from the current config, plus the exact command/edit to apply — and exits without changing anything. With --apply it writes the standard pools into threadpoolworker.properties (backing up the file first) and prints the restart command the admin must run in the change window. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never restarts a running worker itself; it prints the restart step for the admin to perform under change control. Names/flags per the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Remediation.sh # DRY-RUN: prints intended change only ./OC2MBATCH-Remediation.sh --apply # applies (change window only) Override sizing: DEFAULT_THREADS=5 ONLINE_THREADS=3 BILLING_THREADS=8 ./OC2MBATCH-Remediation.sh EXPECTED RESULTS DRY-RUN prints the standard pool line and any drift from the current config, plus the exact command/edit to apply — and exits without changing anything. With --apply it writes the standard pools into threadpoolworker.properties (backing up the file first) and prints the restart command the admin must run in the change window. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never restarts a running worker itself; it prints the restart step for the admin to perform under change control. Names/flags per the OUAF Server Administration Guide — verify for your version.

OC2MBATCH-Report.sql

Purpose: Render the C2M batch monitoring view (C2M Monitor - Batch - Run Status): recent batch run status by control, run durations, thread completion and failures, throughput signals, and the Batch Job Submission backlog. Read-only — SELECT only, it changes no data. Use it during and after the nightly cycle to confirm every control completed and no backlog remains. HOW TO RUN Via SQL*Plus or SQLcl against the C2M schema (read-only account is fine): sqlplus user/pass@your-db @OC2MBATCH-Report.sql (or sql user/pass@your-db @OC2MBATCH-Report.sql with SQLcl) EXPECTED RESULTS Four sections print: (1) runs in the last 24h with status and duration, (2) failed/error threads, (3) currently in-flight runs, (4) pending Batch Job Submission backlog. PASS = all in-scope controls Complete and backlog = 0. FAIL = any Error thread, an overdue in-flight run, or a non-zero backlog. NOTES White-label, repeatable report — the standing operational view, same query set every night. Read-only against live systems. Table/column names follow the OUAF/C2M batch model — verify object names for your version (they are cited, not invented).

How to run: Via SQL*Plus or SQLcl against the C2M schema (read-only account is fine): sqlplus user/pass@your-db @OC2MBATCH-Report.sql (or sql user/pass@your-db @OC2MBATCH-Report.sql with SQLcl) EXPECTED RESULTS Four sections print: (1) runs in the last 24h with status and duration, (2) failed/error threads, (3) currently in-flight runs, (4) pending Batch Job Submission backlog. PASS = all in-scope controls Complete and backlog = 0. FAIL = any Error thread, an overdue in-flight run, or a non-zero backlog. NOTES White-label, repeatable report — the standing operational view, same query set every night. Read-only against live systems. Table/column names follow the OUAF/C2M batch model — verify object names for your version (they are cited, not invented).

OC2MBATCH-ScopeHelper.sh

Purpose: Produce a bounded, reusable scope for the batch work: the list of batch controls to schedule (pilot subset first, then expansion) and the worker nodes they run on. It reads a scope definition file and emits a clean, de-duplicated target list the other scripts and the scheduler consume, so the pilot and each expansion wave run against a known set rather than "everything at once". Read-only — it only reads the scope file. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-ScopeHelper.sh --scope pilot # controls/nodes tagged pilot ./OC2MBATCH-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mbatch-scope.conf) lines: CONTROL,POOL,NODE,SCOPE e.g. BILLING,BILLING,worker-a,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: batch controls with their assigned pool, and the distinct worker nodes. Use the output to drive the scheduler mapping (Implementation Step 4) and to target validation/report runs. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each

expansion wave, so scope is explicit and auditable. Read-only. No live-system changes. Object names (controls, pools) are yours to fill from OC2MBATCH-Discovery.sh output.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-ScopeHelper.sh --scope pilot # controls/nodes tagged pilot ./OC2MBATCH-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mbatch-scope.conf) lines: CONTROL,POOL,NODE,SCOPE e.g. BILLING,BILLING,worker-a,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: batch controls with their assigned pool, and the distinct worker nodes. Use the output to drive the scheduler mapping (Implementation Step 4) and to target validation/report runs. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each expansion wave, so scope is explicit and auditable. Read-only. No live-system changes. Object names (controls, pools) are yours to fill from OC2MBATCH-Discovery.sh output.

OC2MBATCH-Validation.sh

Purpose: Confirm the configured C2M batch estate matches the standard: every worker node advertises the standard thread pools (DEFAULT, ONLINE, BILLING), the batch JMX interface is reachable, and the scheduler submission path responds. Read-only — it validates state and reports PASS/FAIL, it changes nothing. Run it on the pilot (Implementation Step 8) and after each expansion. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Validation.sh Optional: JMX_URL=service:jmx:rmi:///... to probe live-thread JMX on a worker node. EXPECTED RESULTS For each check prints PASS or FAIL: standard pools advertised, JMX summary reachable, REST submit utility present. Overall PASS = every worker offers the standard pool set and JMX/scheduler signals are present. Any FAIL points to the gap to remediate (see OC2MBATCH-Remediation.sh) before re-validating. NOTES White-label, repeatable validation — the same checks every node, every environment. Read-only where it reads live systems (queries JMX with a summary action, reads config). Names/flags are from the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MBATCH-Validation.sh Optional: JMX_URL=service:jmx:rmi:///... to probe live-thread JMX on a worker node. EXPECTED RESULTS For each check prints PASS or FAIL: standard pools advertised, JMX summary reachable, REST submit utility present. Overall PASS = every worker offers the standard pool set and JMX/scheduler signals are present. Any FAIL points to the gap to remediate (see OC2MBATCH-Remediation.sh) before re-validating. NOTES White-label, repeatable validation — the same checks every node, every environment. Read-only where it reads live systems (queries JMX with a summary action, reads config). Names/flags are from the OUAF Server Administration Guide — verify for your version.